S11 · Integración Angular con Keycloak

Jannys Graciela Navarro Acrota

Equipo de referencia: 09 · Casa o Nada. Tarea personal sobre Pagatu.

Aporte documentado: evolución de órdenes, cliente y pago, seguridad, comunicación, verificación y documentación individual con asistencia de herramientas.

Ejecución real: 5 de noviembre de 2026 · America/Lima. Fechas ISO en UTC pueden figurar como 6 de noviembre.

Capturas complementarias de VS Code tomadas el 6 de noviembre de 2026, con reloj real y usuario Windows visibles. Muestran código y archivos originales de las pruebas del 5 de noviembre; visualizar un transcript no constituye una nueva ejecución.

Repositorio GitHub · Stephmarc/Tareas-Distribuidas

Evidencia técnica · cuatro bloques

1. Migración a Keycloak

El estado final usa el realm pagatu y roles ADMIN/CLIENTE de Keycloak. Auth propio se conserva para estudiar S07, pero la SPA y los Resource Servers finales confían en Keycloak. El frontend muestra el usuario real demo-unidad2.

2. Authorization Code y PKCE

Se completó un nuevo login real desde Angular con Authorization Code y PKCE S256. El modo de evidencia comparó el state recibido con el enviado y SHA-256(code_verifier) con code_challenge: ambos true; el intercambio de token respondió HTTP 200. Solo se registran booleanos, fecha y código HTTP. La cuenta cliente es pública, sin client_secret y con Direct access grants desactivado.

3. Estado todavía no resuelto

La SPA crea una orden por el Gateway 18080 y muestra PENDIENTE_PAGO antes del resultado Kafka. Realiza polling cada dos segundos, sin solapar peticiones, con límite de intentos y limpieza del temporizador al destruir el componente.

4. Camino feliz y rechazo

Se detuvo pago-ms y se crearon desde Angular las órdenes 48 y 49 por 31.00. La captura inicial muestra ambas PENDIENTE_PAGO. Después de arrancar pago, la misma página cambió por polling a 48 PAGADA y 49 CANCELADA, sin pulsar Actualizar. Una consulta SQL posterior confirma los dos estados y el mismo clienteId.

Código real del repositorio

main.ts

clients/pagatu-frontend/src/main.ts

```
  1 import {Component,signal,OnDestroy} from '@angular/core';
  2 import {bootstrapApplication} from '@angular/platform-browser';
  3 import Keycloak from 'keycloak-js';
  4 import {rememberAuthorization,observePkce,PkceProof} from './pkce-evidence';
  5 type Producto={id:number,nombre:string,precio:number};
  6 type Orden={id:number,estado:string,total:number|null,detalles:unknown[]};
  7 @Component({selector:'app-root',standalone:true,template:`
  8 <header><strong>pagatu</strong><span>Casa o Nada · Sistemas distribuidos</span><button
(click)="login()">{{authenticated()?'Mi cuenta':'Ingresar'}}</button>@if(authenticated()){<button
(click)="logout()">Salir</button>}</header>
  9 <main>@if(evidence){<p class="notice">Evidencia personal · {{clock()}} · Cuenta:
{{username()}}</p>@if(pkceProof()){<p class="notice">PKCE S256:
{{pkceProof()!.verifierS256Coincide?'verificado':'no coincide'}} · state:
{{pkceProof()!.stateCoincide?'coincide':'no coincide'}} · intercambio HTTP {{pkceProof()!.http}}</p>}}<p
class="eyebrow">UNIDAD 2 · CLIENTE DE ÓRDENES</p><h1>Productos y mis pedidos</h1><p>Consulta el catálogo,
 registra una orden y sigue el resultado del pago.</p>
 10 @if(message()){<p class="notice" role="status">{{message()}}</p>}
 11 @if(!authenticated()){<section><h2>Inicia sesión para continuar</h2><p>Tu cuenta permite consultar
tus órdenes y su estado de pago.</p><button (click)="login()">Ingresar con Keycloak</button></section>}
 12 @if(authenticated()){
 13 <section><h2>Catálogo</h2><div class="products">@for(p of products();track
p.id){<article><h3>{{p.nombre}}</h3><p>S/ {{p.precio.toFixed(2)}}</p><label>Cantidad <input #cantidad
type="number" min="1" max="100" value="1"></label><label>Método <select #metodo><option
value="YAPE_PLIN">Yape / Plin</option><option value="RECHAZADO">Prueba de
rechazo</option></select></label><button [disabled]="busy()"
(click)="create(p.id,+cantidad.value,metodo.value)">Crear orden</button></article>}</div></section>
 14 <section><div class="row"><h2>Mis pedidos</h2><button
(click)="loadOrders()">Actualizar</button></div><p>Un pedido pendiente aún no tiene un resultado de pago
confirmado.</p><table><thead><tr><th>Orden</th><th>Estado</th><th>Total</th></tr></thead><tbody>@for(o of
 orders();track o.id){<tr><td>#{{o.id}}</td><td><span
class="badge">{{o.estado}}</span></td><td>{{o.total===null?'Por confirmar':'S/
'+o.total.toFixed(2)}}</td></tr>}@empty{<tr><td colspan="3">Todavía no hay
pedidos.</td></tr>}</tbody></table></section>}
 15 </main><footer>Jannys Graciela Navarro Acrota · Casa o Nada</footer>`})
 16 class App implements OnDestroy{
 17  pkceProof=signal<PkceProof|null>(null);evidence=new
URLSearchParams(location.search).has('evidencia');clock=signal(new Date().toLocaleString('es-
PE',{timeZone:'America/Lima',hour12:false}));username=signal('Sin sesión');private
clockTimer=setInterval(()=>this.clock.set(new Date().toLocaleString('es-
PE',{timeZone:'America/Lima',hour12:false})),1000);
 18  authenticated=signal(false);products=signal<Producto[]>([]);orders=signal<Orden[]>([]);message=signa
l('');busy=signal(false);
 19  private keycloak=new Keycloak({url:'http://localhost:18090',realm:'pagatu',clientId:'pagatu-
frontend'});
 20  private timer:ReturnType<typeof setInterval>|undefined;private attempts=0;private polling=false;
 21  constructor(){if(this.evidence)observePkce(proof=>this.pkceProof.set(proof));this.keycloak.init({onL
oad:'check-sso',pkceMethod:'S256',checkLoginIframe:false}).then(async
ok=>{this.authenticated.set(ok);this.username.set(this.keycloak.tokenParsed?.['preferred_username']||'Sin
 sesión');if(ok){await this.loadProducts();await this.loadOrders();}}).catch(()=>this.message.set('No se
pudo conectar con el servicio de acceso. Comprueba que Keycloak esté iniciado.'));}
 22  async login(){if(this.evidence){const url=await this.keycloak.createLoginUrl({redirectUri:location.o
rigin+'/?evidencia=1',scope:'openid'});rememberAuthorization(url);location.assign(url);return;}return
this.keycloak.login({redirectUri:location.origin,scope:'openid'});}
 23  logout(){return
this.keycloak.logout({redirectUri:location.origin+(this.evidence?'/?evidencia=1':'')});}
 24  private async api<T>(url:string,options:RequestInit={}):Promise<T>{await
this.keycloak.updateToken(30);const response=await
fetch('http://localhost:18080/api/v1/'+url,{...options,headers:{'Content-
Type':'application/json',Authorization:'Bearer '+this.keycloak.token}});if(!response.ok)throw new
Error('La operación respondió HTTP '+response.status);return response.json() as Promise<T>;}
 25  async loadProducts(){try{this.products.set(await
this.api<Producto[]>('productos'));}catch(e){this.message.set(String(e));}}
 26  async create(id:number,cantidad:number,metodoPago:string){if(this.busy()||!Number.isInteger(cantidad
)||cantidad<1){return;}this.busy.set(true);try{const o=await this.api<Orden>('ordenes',{method:'POST',bod
y:JSON.stringify({metodoPago,detalles:[{idProducto:id,cantidad}]})});this.message.set(o.estado==='CARRITO
'?'Orden guardada. El catálogo no confirmó los precios.':'Orden registrada. Consulta su estado en Mis
pedidos.');await this.loadOrders();}catch(e){this.message.set(String(e));}finally{this.busy.set(false);}}
 27  async loadOrders(){try{this.orders.set(await this.api<Orden[]>('ordenes'));if(this.orders().some(o=>
o.estado==='PENDIENTE_PAGO')){if(!this.timer){this.attempts=0;this.timer=setInterval(()=>{if(this.polling
)return;if(++this.attempts>120){this.stop();this.message.set('El pago sigue pendiente. Puedes volver a
consultar más tarde.');return;}this.polling=true;this.loadOrders().finally(()=>this.polling=false);},2000
);}}else{this.stop();}}catch(e){this.message.set(String(e));this.stop();}}
 28  private stop(){if(this.timer)clearInterval(this.timer);this.timer=undefined;}
 29  ngOnDestroy(){this.stop();clearInterval(this.clockTimer);}
 30 }
 31 bootstrapApplication(App).catch(console.error);
```

pkce-evidence.ts

clients/pagatu-frontend/src/pkce-evidence.ts

```
  1 // Diagnostic mode observes the real exchange; it never stores tokens or code_verifier.
  2 export type PkceProof={stateCoincide:boolean,verifierS256Coincide:boolean,http:number,fecha:string};
  3 const key='pagatu-evidence-oauth';
  4 export function rememberAuthorization(url:string){
  5  const u=new URL(url);
  6  sessionStorage.setItem(key,JSON.stringify({state:u.searchParams.get('state'),challenge:u.searchParam
s.get('code_challenge')}));
  7 }
  8 export function observePkce(done:(proof:PkceProof)=>void){
  9  const saved=sessionStorage.getItem(key);if(!saved)return;
 10  const expected=JSON.parse(saved) as {state:string,challenge:string};
 11  const returnedState=new URLSearchParams(location.hash.slice(1)).get('state');
 12  const original=window.fetch.bind(window);
 13  window.fetch=async(input:RequestInfo|URL,init?:RequestInit)=>{
 14   const url=typeof input==='string'?input:input instanceof URL?input.href:input.url;
 15   const body=init?.body;
 16   if(url==='http://localhost:18090/realms/pagatu/protocol/openid-connect/token'&&body instanceof
URLSearchParams&&body.get('grant_type')==='authorization_code'){
 17    window.fetch=original;
 18    const verifier=body.get('code_verifier')||'';
 19    const digest=new Uint8Array(await crypto.subtle.digest('SHA-256',new
TextEncoder().encode(verifier)));
 20    const
challenge=btoa(String.fromCharCode(...digest)).replace(/\+/g,'-').replace(/\//g,'_').replace(/=+$/,'');
 21    const response=await original(input,init);
 22    const proof={stateCoincide:returnedState===expected.state,verifierS256Coincide:!!verifier&&challen
ge===expected.challenge,http:response.status,fecha:new Date().toISOString()};
 23    sessionStorage.removeItem(key);done(proof);console.info('PKCE_EVIDENCE '+JSON.stringify(proof));
 24    return response;
 25   }
 26   return original(input,init);
 27  };
 28 }
```

Ejecución y resultados originales

Comando reproducible: node scripts/Verify-Frontend.cjs

Los transcripts son salida real de PowerShell y los JSON provienen de las respuestas de la aplicación. Se adjuntan completos en MkDocs. No se sustituyen por un texto de aprobación.

Inicio de sesion real con Keycloak: Authorization Code, state y code_challenge_method=S256 visibles en URL de autorizacion.

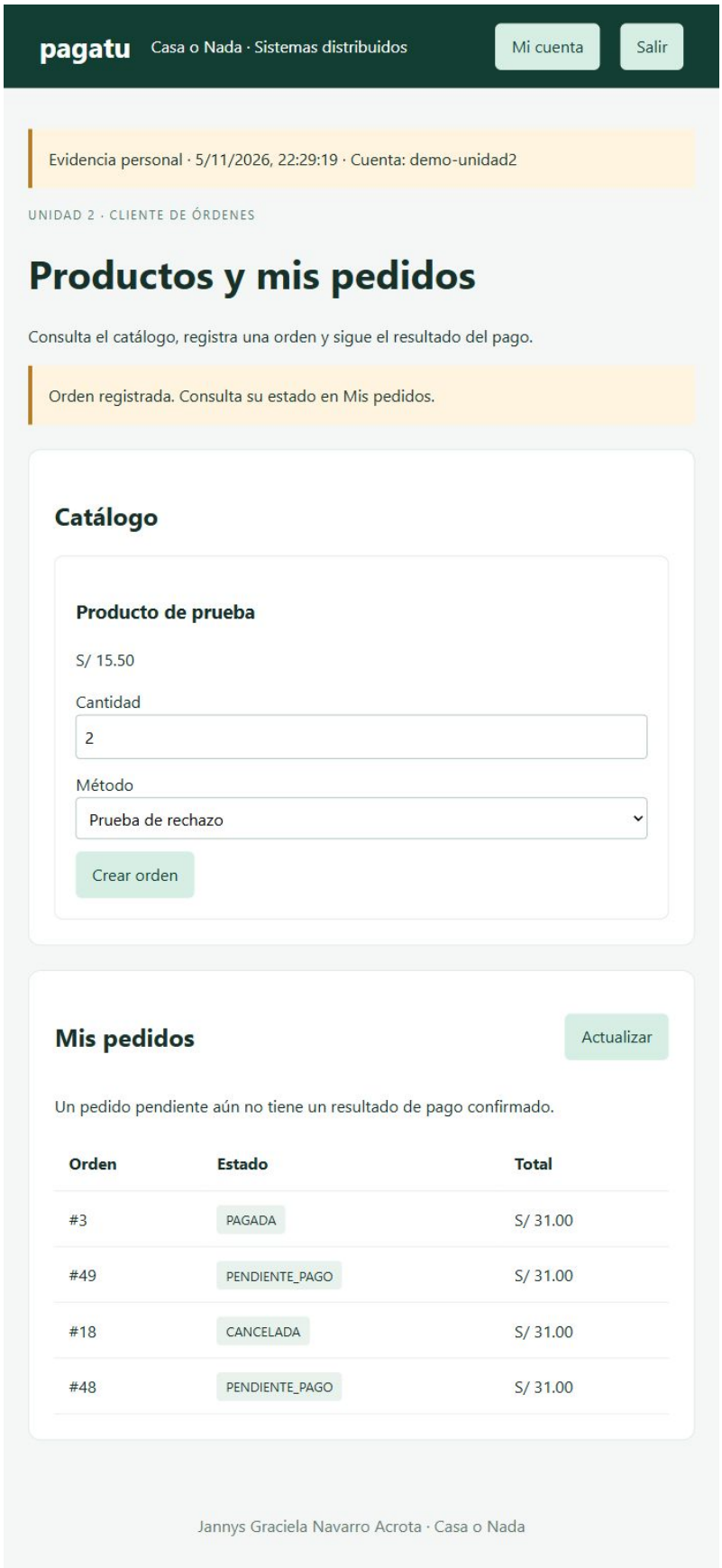
Pago detenido: UI crea ordenes 48 y 49 y muestra PENDIENTE_PAGO.

Pago recuperado: polling de 2 segundos actualiza 48 PAGADA y 49 CANCELADA sin pulsar Actualizar.

SELECT id,cliente_id,estado,total FROM ordenes WHERE id IN (48,49) ORDER BY id

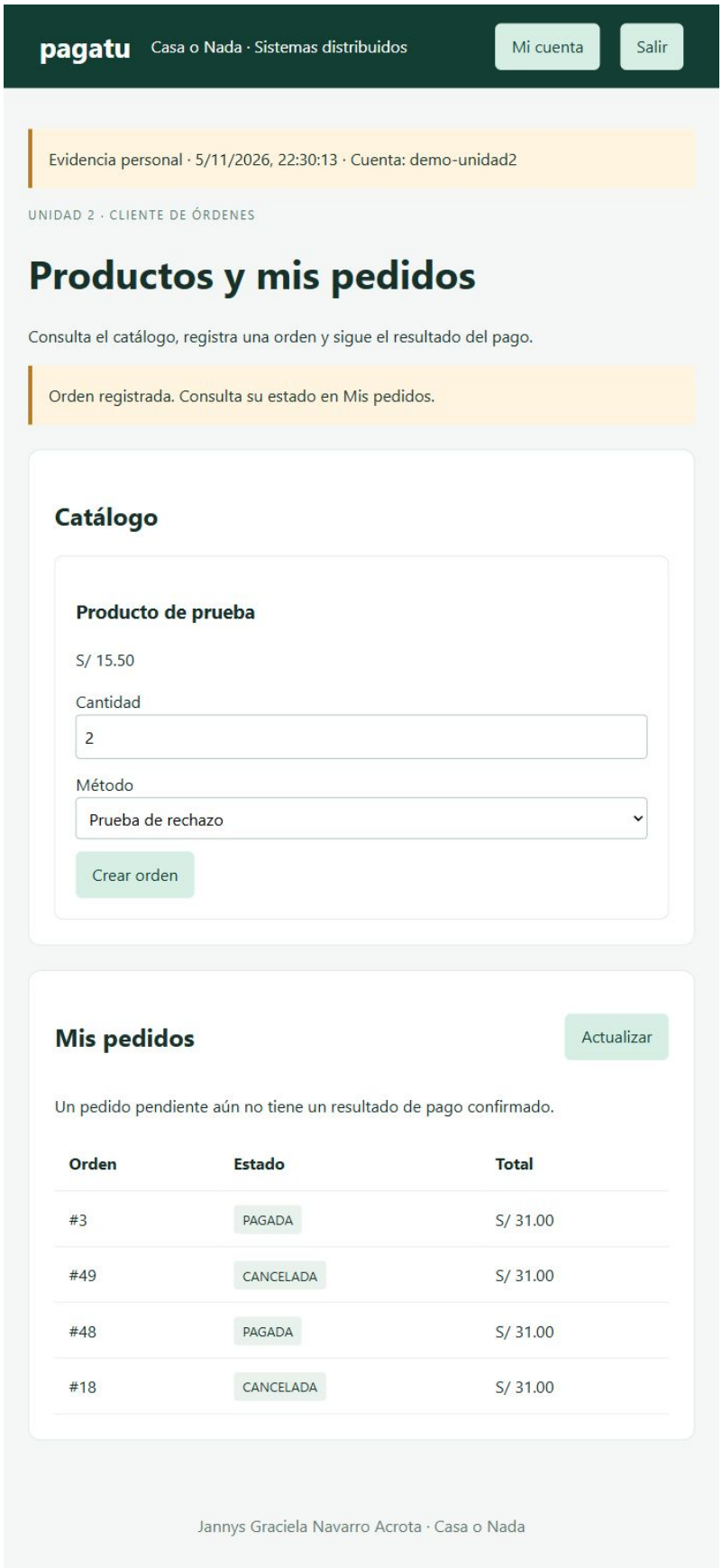
48|bbd5e8a1-00fd-4b2f-ace7-0d1df66f8968|PAGADA|31.00
49|bbd5e8a1-00fd-4b2f-ace7-0d1df66f8968|CANCELADA|31.00

Captura real



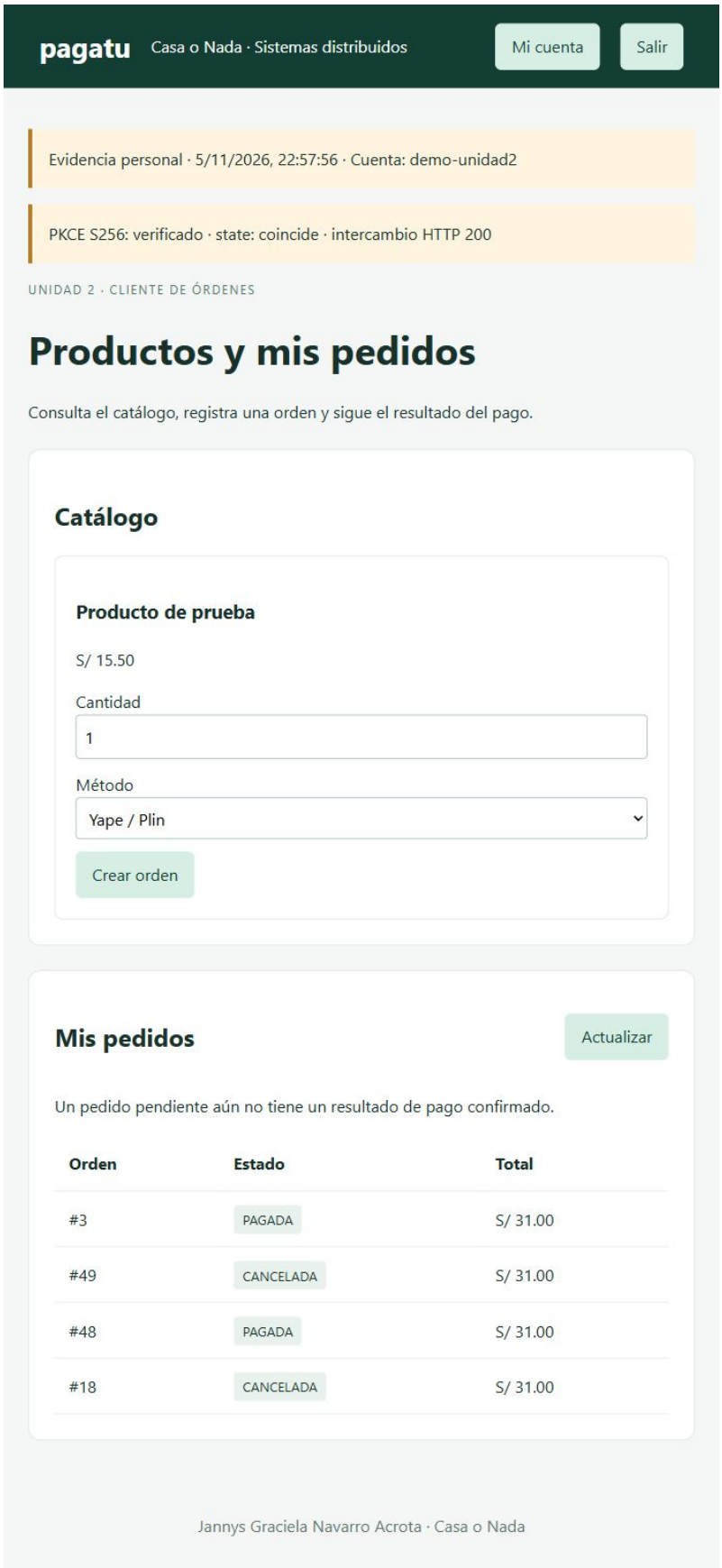
Captura íntegra, sin edición de resultados ni fechas. Angular con pago-ms detenido: órdenes 48 y 49 PENDIENTE_PAGO. Hora real y cuenta demo-unidad2 visibles.. La salida original completa está vinculada en MkDocs.

Captura real



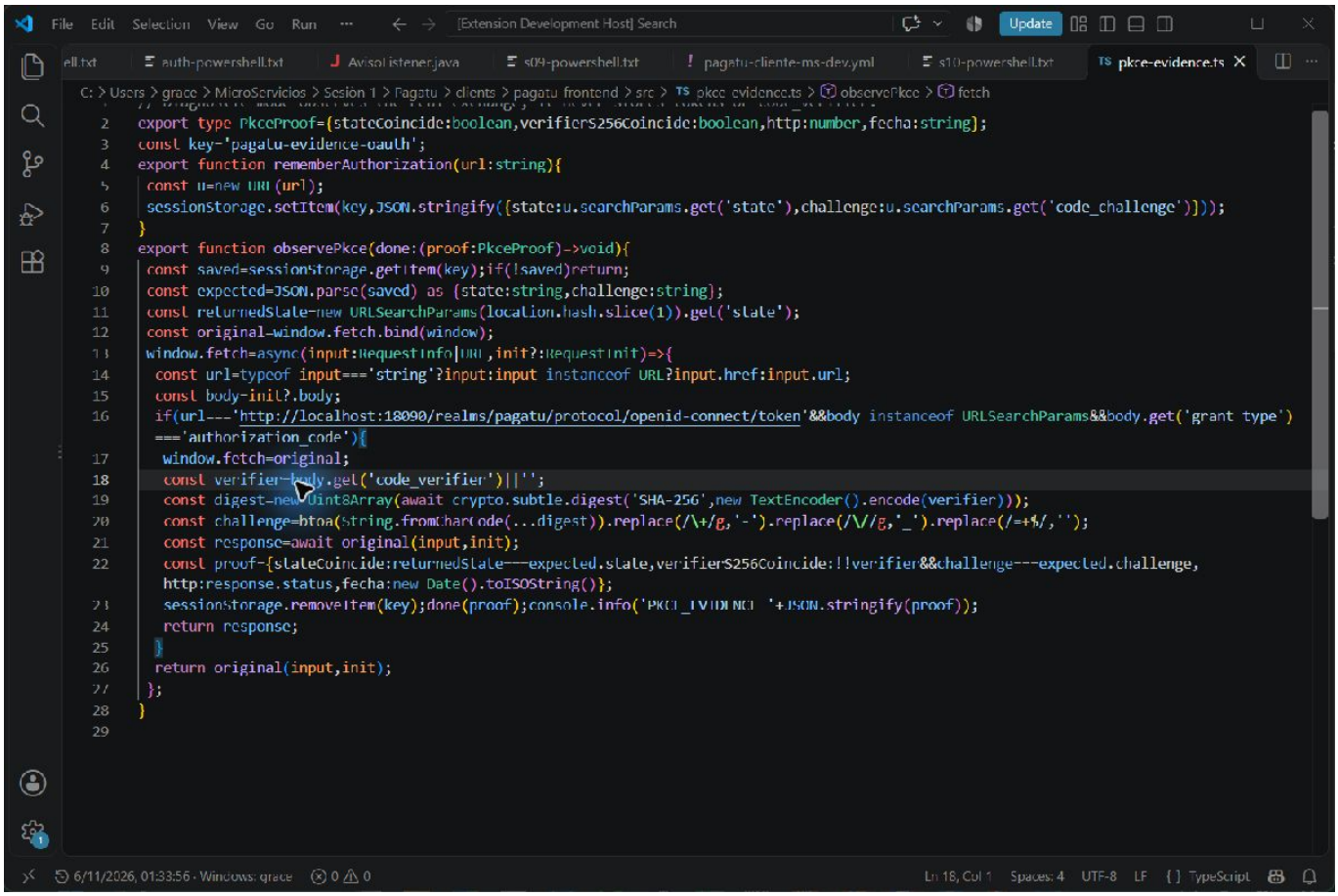
Captura íntegra, sin edición de resultados ni fechas. La misma página después de recuperar pago-ms: 48 PAGADA y 49 CANCELADA, recibidas por polling sin pulsar Actualizar.. La salida original completa está vinculada en MkDocs.

Captura real



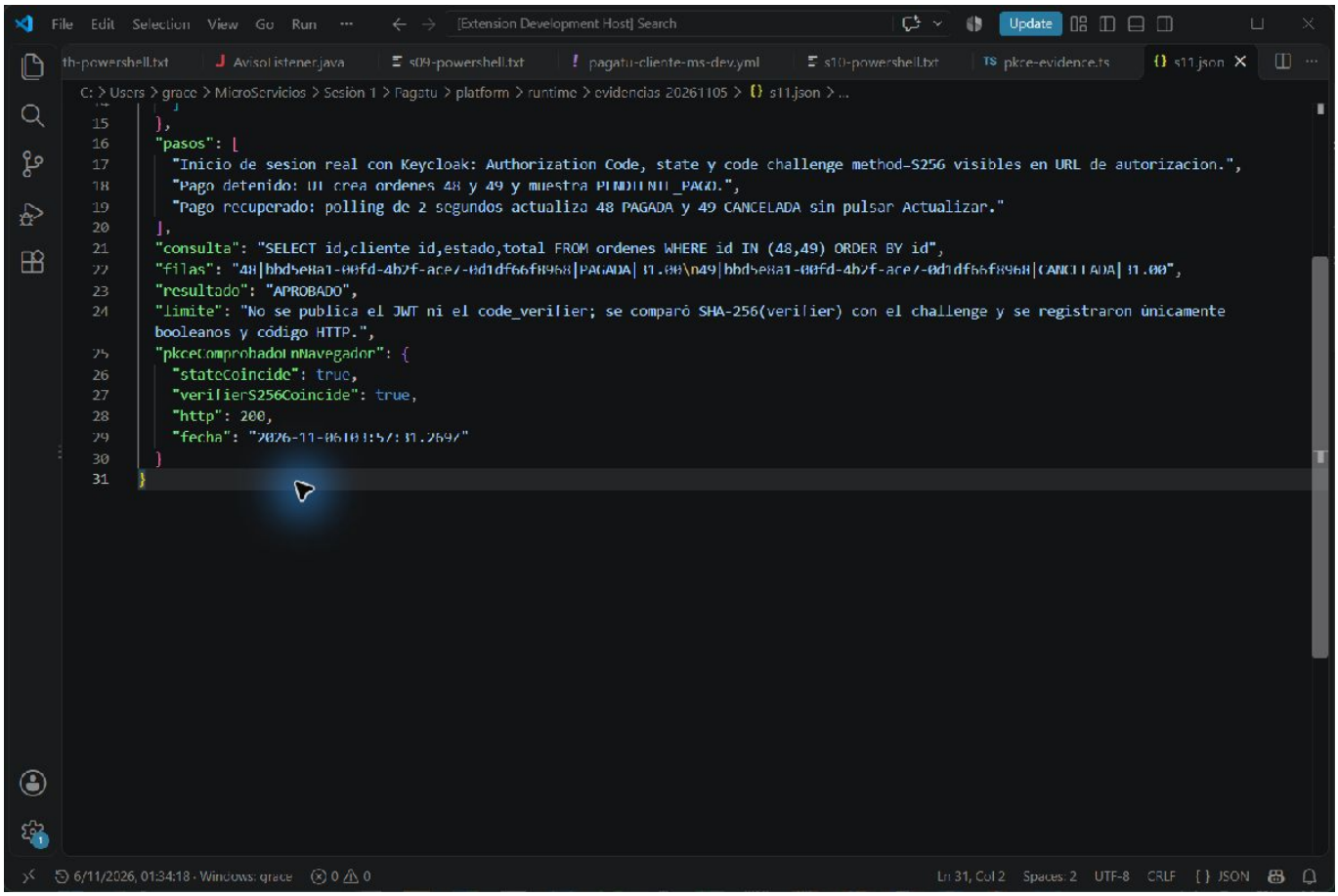
Captura íntegra, sin edición de resultados ni fechas. Login real: state coincide, SHA-256 del verifier coincide con el challenge y el intercambio HTTP responde 200. No se muestran tokens ni verificadores.. La salida original completa está vinculada en MkDocs.

Captura real



Captura íntegra, sin edición de resultados ni fechas. Código real: compara state y SHA-256(code_verifier) con code_challenge; registra únicamente booleanos y estado HTTP.. La salida original completa está vinculada en MkDocs.

Captura real



Captura íntegra, sin edición de resultados ni fechas. Reporte original: SQL confirma 48 PAGADA y 49 CANCELADA; stateCoincide=true, verifierS256Coincide=true e intercambio HTTP 200.. La salida original completa está vinculada en MkDocs.

Hallazgo y reflexión técnica

Error o hallazgo real

Durante las pruebas hubo un catálogo temporalmente vacío hasta completar la carga asíncrona tras recuperar servicios. La interfaz conserva la distinción entre sin resultados, error y pago pendiente. No presenta una creación HTTP 201 como pago completado.

Reflexión técnica

La respuesta de creación confirma la orden local, no el resultado del pago.

Por eso la interfaz mantiene PENDIENTE_PAGO sin anunciar éxito definitivo.

El polling consulta el estado autoritativo por el Gateway cada dos segundos.

La UI termina la espera al recibir PAGADA o CANCELADA.

El límite de intentos evita consultar indefinidamente si no llega el evento.

PKCE permite una SPA pública sin almacenar un client_secret.

sub identifica al comprador; el precio y el resultado del pago los decide el backend.

Límite de alcance: esta entrega personal usa Pagatu; la adaptación al Proyecto Sello grupal CASAS se evalúa por separado. No se atribuye una defensa oral realizada ni una nota de evaluación.

Anexo · Feedback de la sesión

Campos personales pendientes de completar por Jannys. No se inventan respuestas sobre la experiencia, comprensión, esfuerzo o satisfacción en clase.

1. ¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?

2. ¿Qué punto de la clase te resultó más confuso o te dejó con dudas?

3. ¿Tienes alguna pregunta para la siguiente clase?

4. Nivel de comprensión: Entendido / Más o menos / Necesito ayuda.

5. ¿Cómo puede ayudarte el docente a comprender mejor el tema?

6. Participación: Muy comprometido/a / Comprometido/a / Poco comprometido/a.

7. Satisfacción con la clase: 1 a 10.